

Università Politecnica delle Marche

Corso di Laurea Magistrale

Ingegneria Informatica e dell'Automazione



Sistemi Operativi Dedicati

Studenti

Accattoli Eleonora - S1106361

Brugiavini Alessio - S1106565

Anno Accademico 2022/2023

Indice

1	Introduzione	5
2	Strumenti e software utilizzati	6
3	Sviluppo Progetto	7
3.1	Collegamento dispositivi	7
3.2	Configurazione Raspberry	8
3.3	FreeRTOS	14
3.4	ESP32, MQTT Broker e MQTT Client	17
3.5	Sensori	19
3.5.1	Sensor Task	19
3.5.2	MQTT Task	20
3.5.3	SyncTimeTask	24
3.6	Bot Telegram	24
3.7	Creazione Database e caricamento dati	27
3.8	Grafici	33
4	Considerazioni finali	38

Elenco delle figure

1	Struttura progetto	5
2	Schema di cablatura del progetto	8
3	Raspberry Pi Imager	9
4	Abilitazione SSH	10
5	Modifica connessione	10
6	Configurazione IP statico	11
7	Visualizzazione IP	11
8	Visualizzazione IP da terminale	12
9	File Service	14
10	Stato MQTT Broker	14
11	Alert superamento soglia temperatura	23
12	Alert superamento soglia pressione	23
13	Bot Telegram	27
14	GUI Xampp	28
15	Pressione inviata al broker	30
16	Database	31
17	Comando Bot "leggi"	33
18	Messaggio di errore	33
19	Grafico andamento temperatura	35
20	Grafico andamento pressione	35
21	Grafico andamento luminosità	36

Listings

1	Creazione Task	15
2	Rilevazione sensori e sincronizzazione data e ora	16
3	Inizializzazione sensori	16
4	NTP Client	16
5	Connessione WiFi	16
6	Dichiarazione costanti e variabili globali	17
7	Connessione MQTT Broker	18
8	Connessione WiFi e MQTT Broker	18
9	Sottoscrizione topic	19
10	Struct SensorData	19
11	Task Lettura dati	20
12	Invio dati alla coda	20
13	Ricezione e pubblicazione dati	21
14	Variabili e costanti per monitoraggio soglia	21
15	MQTT Task	22
16	Task sincronizzazione data e ora	24
17	Sincronizzazione NTP	24
18	Codice identificativo bot Telegram	24
19	Funzione invio messaggi Telegram	25
20	Funzione ricezione messaggi Telegram e aggiunta chat ID	26
21	Dati configurazione database	28
22	Connessione al database	28
23	Funzione per configurare e avviare client MQTT	29
24	Comando start	31
25	Comando leggi con estrazione dati	32
26	Comando leggi con estrazione dati	32
27	Calcolo statistiche	33
28	Definizione istruzioni main	37

1 Introduzione

L'obiettivo del progetto consiste nel creare un sistema in grado di rilevare e comunicare in tempo reale i dati rilevati di temperatura, pressione, umidità e luminosità dai sensori nell'ambiente. Il sistema sarà composto da:

- **Raspberry Pi 4B**
- **ESP32**: microcontrollore utilizzato per la progettazione di dispositivi connessi e di sistemi IoT.
- Sensore di temperatura e pressione **BMP280**
- Sensore di luminosità **BH1750**
- Modulo RTC **PCF8523**
- **Broker e Client MQTT**
- **Bot Telegram**

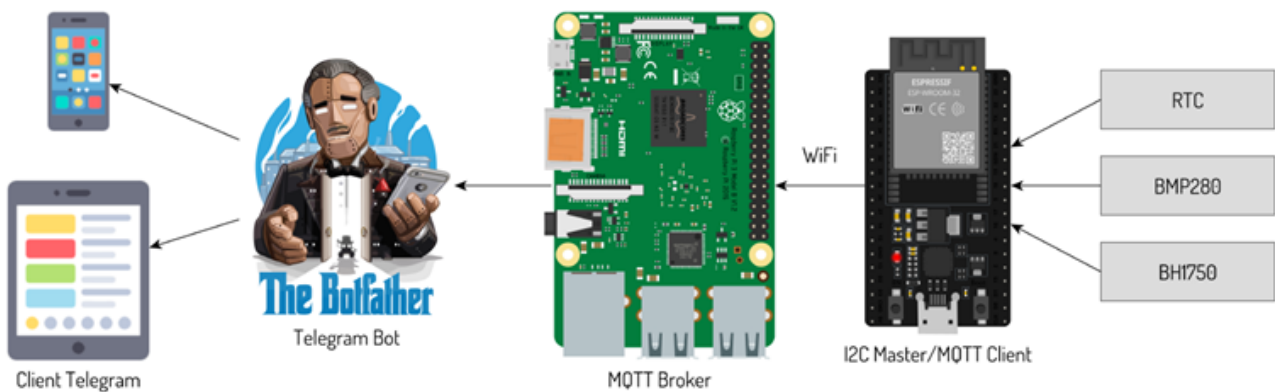


Figura 1: Struttura progetto

I due sensori ed il modulo RTC vengono collegati fisicamente all'ESP32, il quale si occuperà di trasmettere i dati rilevati dai sensori al Raspberry Pi, mediante WiFi. L'ESP32 è configurato come MQTT Client con l'utilizzo del kernel FreeRTOS, il quale permette il coordinamento dei vari task. Il Raspberry Pi sarà configurato come server IoT, il quale svolgerà le funzioni di MQTT Broker, storage dati ed invio alert.

2 Strumenti e software utilizzati

Per lo sviluppo del progetto sono stati utilizzati i seguenti strumenti:

- **Raspberry Pi 4B:** computer a scheda singola basato sul sistema operativo Linux che, opportunamente configurato, permette l'utilizzo in vari campi dell'automazione. Si basa su un system-on-a-chip, ovvero è formato da un circuito integrato che in un solo chip contiene un intero sistema e utilizza una micro SD per il boot e per la memoria ROM non volatile. All'interno del Raspberry Pi è possibile installare, tramite apposito software, un qualsiasi sistema operativo.
- **BMP280:** sensore di temperatura, umidità e pressione.
- **BH1750:** sensore di luminosità.
- **Modulo RTC PCF8523:** è un real-time clock, ovvero un dispositivo con funzione di orologio.

Mentre i software utilizzati sono i seguenti:

- **Visual Studio Code:** editor di codice sorgente sviluppato da Microsoft per Windows, Linux e Mac OS. Permette, tramite apposita estensione, la programmazione di dispositivi remoti mediante connessione SSH.
- **FreeRTOS:** sistema operativo open source in tempo reale per microcontrollori e microprocessori. Verrà utilizzato nell'ESP32.
- **Mosquitto:** viene utilizzato come broker MQTT open-source all'interno del Raspberry Pi. Il broker MQTT è responsabile della ricezione e del filtraggio dei messaggi, dell'identificazione dei client iscritti e l'invio dei messaggi ad essi.
- **Raspberry Pi OS:** è il sistema operativo installato sul Raspberry Pi.

3 Sviluppo Progetto

3.1 Collegamento dispositivi

I dispositivi per poter funzionare e lavorare tra loro necessitano di un cablaggio su una bread-board.

Sono stati collegati i seguenti componenti:

- **BMP280:** sensore di temperatura e pressione.
- **BH1750:** sensore di luminosità.
- **Modulo RTC PCF8523:** è un real-time clock, ovvero un dispositivo con funzione di orologio.
- **ESP32:** microcontrollore nel quale caricare il codice per gestire i vari task.

Il cablaggio è stato effettuato considerando lo schema seguente:

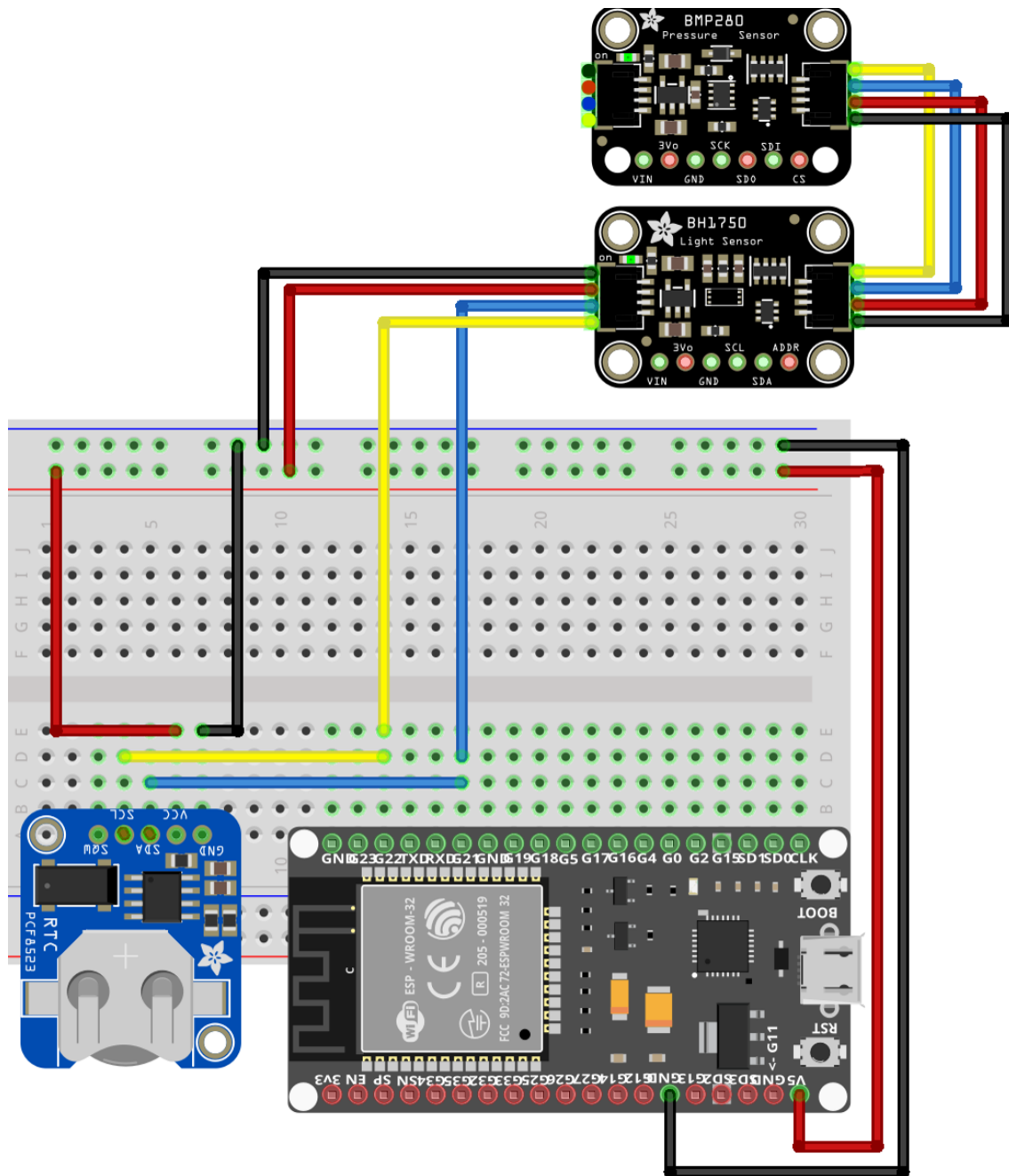


Figura 2: Schema di cablaggio del progetto

In seguito l'ESP32 è stato collegato al PC utilizzando un cavo Micro USB.

3.2 Configurazione Raspberry

Per poter utilizzare il Raspberry è stato necessario installare all'interno della sua scheda SD il sistema operativo dedicato, attraverso il software Raspberry Pi Imager. Come si può vedere dall'immagine sottostante, dalla schermata iniziale, si sceglie il modello del Raspberry, il sistema operativo da installare e la periferica su cui farlo; nel nostro caso, è stato scelto il modello Raspberry Pi 4B, il Raspberry Pi OS e la scheda SD.



Figura 3: Raspberry Pi Imager

Dopodiché viene inserita la scheda SD sul Raspberry, vengono collegati i dispositivi necessari, quali display, tastiera, mouse e cavo Ethernet e vengono impostate le credenziali (nome utente e password).

Il protocollo **SSH** o Secure Shell permette una comunicazione client-server sicura tra dispositivi.

Per configurare la connessione SSH, viene utilizzato **PuTTY**, che è un client SSH gratuito portatile che permette la gestione dei dispositivi da remoto.

PuTTY viene installato nella macchina su cui si intende lavorare da remoto (nel nostro caso una macchina Windows), specificando sotto il campo **Host Name (or IP address)** il nome del nostro Raspberry e si seleziona come **Connection type** l'opzione **SSH**.

Dopo aver cliccato “**Open**”, verranno inserite le credenziali di Raspberry sulla finestra visualizzata.

In questo modo è stata instaurata una connessione SSH tra il PC ed il dispositivo.

Per effettuare la programmazione del Raspberry direttamente dal PC, è necessario installare l'estensione **REMOTE-SSH** su Visual Studio Code, aggiungere una nuova macchina ed inserire i dati della connessione instaurata:

- **nome utente:** nome Raspberry Pi
- **host:** nome host (da terminale inviare comando **hostname**)
- **nome utente:** nel nostro caso **brugia**

Dopodiché come conferma, si dovrà inserire la password impostata nel Raspberry.

Dalle impostazioni del Raspberry, sulla sezione **Interfacce**, viene abilitato il servizio SSH, come da figura sottostante.

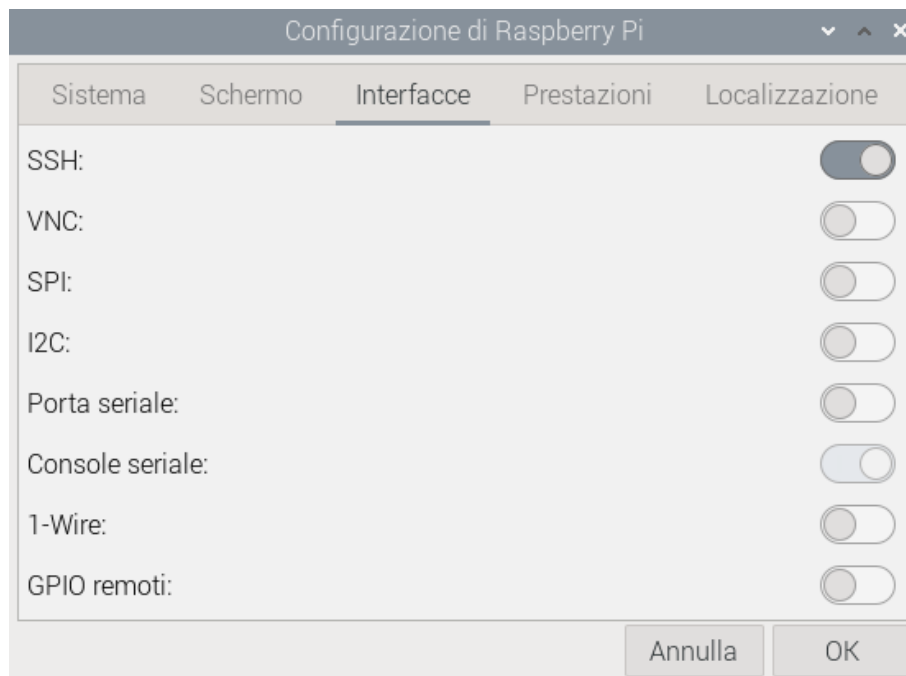


Figura 4: Abilitazione SSH

Essendo il Raspberry connesso alla rete ha un proprio indirizzo IP, che nel nostro caso è: **192.168.1.12**. Per assegnare al Raspberry un indirizzo IP statico, bisogna:

1. Inviare da terminale il seguente comando per modificare la connessione:

```
1 sudo nmtui edit "Nome connessione"
```

```
brugia@raspberrypi:~ $ sudo nmtui edit TIM-63619464
brugia@raspberrypi:~ $
```

Figura 5: Modifica connessione

2. Nella sezione **CONFIGURAZIONE IPv4** dell'editor che si è aperto, cambiare l'opzione **Automatico** in **Manuale**
3. Cliccare su **Mostra** ed inserire sotto il campo **Indirizzi**, l'indirizzo IP statico da assegnare al dispositivo.
4. Cliccare **OK** per salvare la configurazione.
5. Riavviare il servizio di rete, inviando da terminale:

```
sudo systemctl restart NetworkManager
```

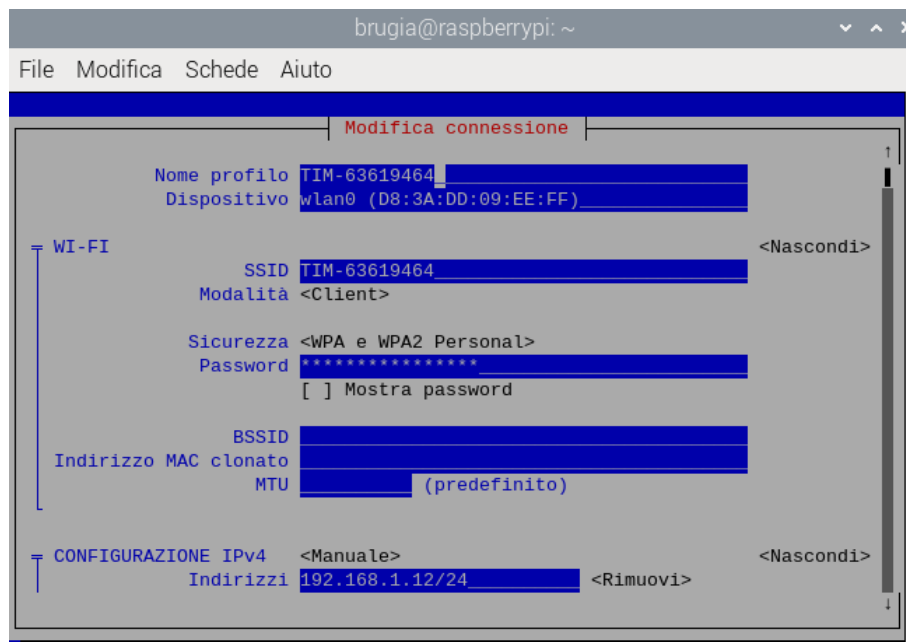


Figura 6: Configurazione IP statico

Per verificare l'indirizzo IP assegnato al Raspberry si può trascinare il mouse sopra l'icona di rete oppure da terminale basta inviare come comando **hostname -I**.

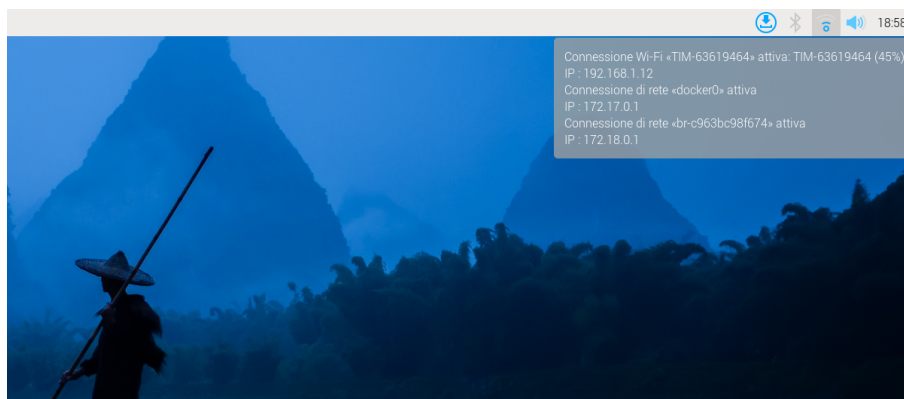


Figura 7: Visualizzazione IP

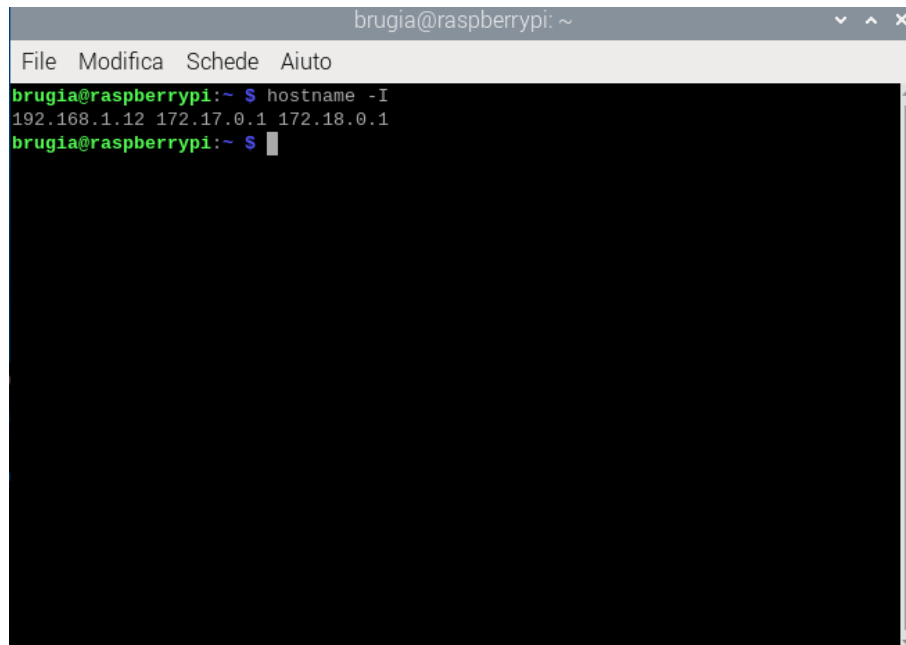


Figura 8: Visualizzazione IP da terminale

Il Raspberry Pi dovrà svolgere la funzione di server IoT, ovvero si occuperà di svolgere la funzione di MQTT Broker, storage dati ed invio alert.

L'**MQTT** è un protocollo di messaggistica standard per l'IoT, che, attraverso un broker, consente lo scambio di messaggi tra dispositivi.

Il broker MQTT è il sistema backend, responsabile della ricezione e del filtraggio dei messaggi, della decisione di chi è interessato a loro e della pubblicazione del messaggio a tutti i client iscritti.

Il client MQTT è un qualsiasi dispositivo che utilizza il servizio MQTT.

Tra i vari broker disponibili, viene usato **Mosquitto**, che è il più comune, il quale è stato installato sul Raspberry tramite VS Code, con i seguenti passaggi:

1. Viene aggiornato il sistema:

```
sudo apt update && sudo apt upgrade
```

2. Si installa Mosquitto :

```
sudo apt install -y mosquitto mosquitto-clients
```

3. Si lancia il comando per avviare Mosquitto automaticamente all'avvio:

```
sudo systemctl enable mosquitto.service
```

4. Si testa l'installazione:

```
mosquitto -v
```

Viene poi abilitato l'accesso remoto senza autenticazione:

1. Si apre il file **mosquitto.conf**

```
sudo nano /etc/mosquitto/mosquitto.conf
```

2. Si aggiungono alla fine del file le seguenti istruzioni per configurare l'accesso non autenticato:

```
listener 1883
allow_anonymous true
```

3. Si salva il file e si riavvia Mosquitto.

Per far avviare in automatico lo script Python è necessario creare un file **service** all'interno della cartella **systemd** del Raspberry. Il file è composto da tre sezioni:

1. **Unit:**

- **Description:** nome servizio
- **After:** determina quando inizia il servizio.

2. **Service:**

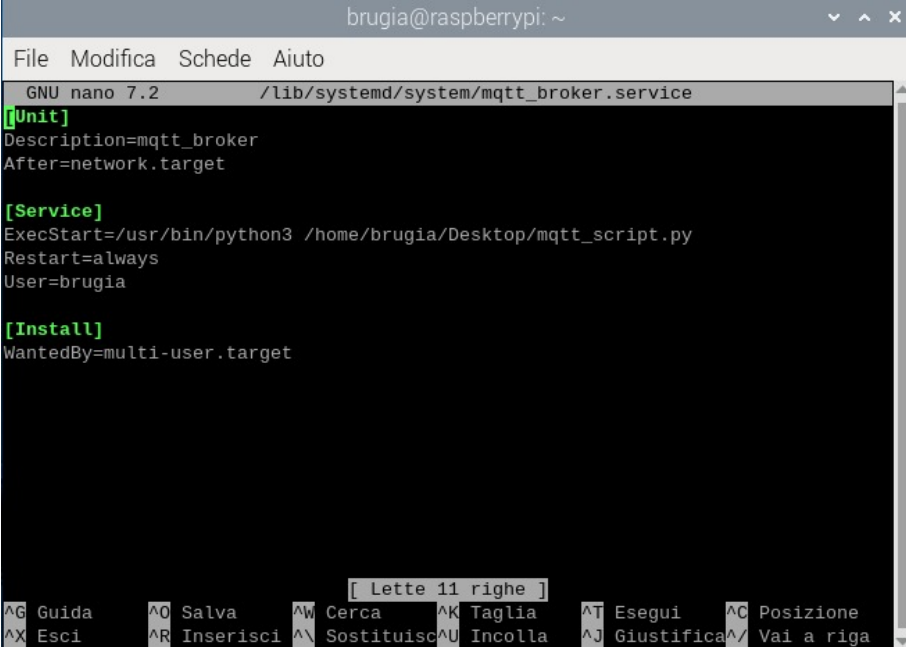
- **ExecStart:** contiene il percorso dello script che deve essere eseguito in automatico all'avvio.
- **Restart:** indica se il servizio deve essere riavviato in caso di uscita.
- **User:** identifica il nome del Raspberry.

3. **Install:**

- **WantedBy:** specifica l'attivazione di un'unità. Impostandola su **multi-user.target**, si può stabilire una relazione di dipendenza che eseguirà il programma quando il controllo del sistema passa alla shell multiutente.

Nel nostro caso, viene fatto eseguire lo script che si occupa della connessione all'MQTT broker e dell'invio dei dati rilevati dai sensori.

Nella figura sottostante si può vedere il file **service** che è stato creato all'interno della cartella **systemd** con le relative informazioni.



```
brugia@raspberrypi: ~  
File Modifica Schede Aiuto  
GNU nano 7.2 /lib/systemd/system/mqtt_broker.service  
[Unit]  
Description=mqtt_broker  
After=network.target  
  
[Service]  
ExecStart=/usr/bin/python3 /home/brugia/Desktop/mqtt_script.py  
Restart=always  
User=brugia  
  
[Install]  
WantedBy=multi-user.target  
  
[ Lette 11 righe ]  
^G Guida ^O Salva ^W Cerca ^K Taglia ^T Esegui ^C Posizione  
^X Esci ^R Inserisci ^_ Sostituisci ^U Incolla ^J Giustifica ^_ Vai a riga
```

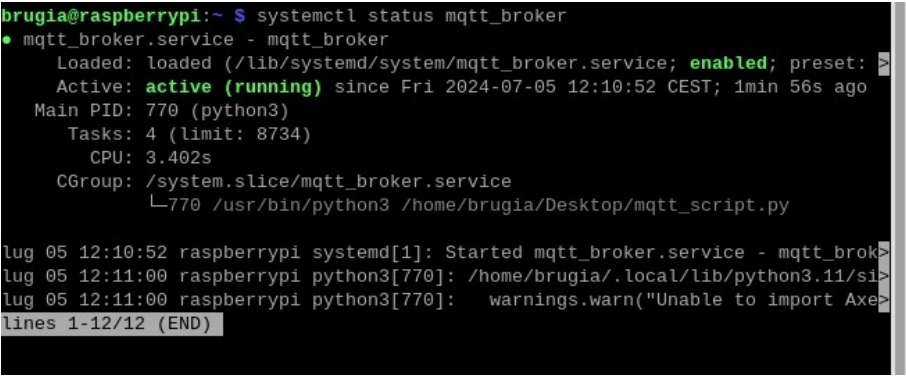
Figura 9: File Service

Una volta salvato il file, si effettua il riavvio del dispositivo.

Per verificare che il servizio venga effettivamente eseguito in automatico, si invia da terminale il comando:

```
systemctl status mqtt_broker
```

e si controlla se lo stato di **Active** sia **active (running)**, come da figura sottostante.



```
brugia@raspberrypi:~$ systemctl status mqtt_broker  
● mqtt_broker.service - mqtt_broker  
   Loaded: loaded (/lib/systemd/system/mqtt_broker.service; enabled; preset: >  
   Active: active (running) since Fri 2024-07-05 12:10:52 CEST; 1min 56s ago  
   Main PID: 770 (python3)  
     Tasks: 4 (limit: 8734)  
        CPU: 3.402s  
   CGroup: /system.slice/mqtt_broker.service  
           └─770 /usr/bin/python3 /home/brugia/Desktop/mqtt_script.py  
  
lug 05 12:10:52 raspberrypi systemd[1]: Started mqtt_broker.service - mqtt_brok>  
lug 05 12:11:00 raspberrypi python3[770]: /home/brugia/.local/lib/python3.11/si>  
lug 05 12:11:00 raspberrypi python3[770]: warnings.warn("Unable to import Axe>  
lines 1-12/12 (END)
```

Figura 10: Stato MQTT Broker

3.3 FreeRTOS

FreeRTOS è un sistema operativo real-time in grado di gestire più attività in parallelo e di rispettare i tempi di esecuzione dei processi.

Per fare ciò vengono utilizzati i **task**, ovvero delle funzioni contenenti una serie di istruzioni da eseguire in loop.

Per lo svolgimento del progetto, sono stati creati ed implementati i seguenti task:

- **sensorTask**: viene usato per leggere i dati dai sensori BMP280 e BH1750, quali temperatura, pressione e luminosità.
- **mqttTask**: viene usato per instaurare la connessione MQTT.
- **syncTimeTask**: viene usato per sincronizzare i dati del sensore RTC, quali data e ora con quelli del server NTP.

Per creare i task vengono utilizzate le seguenti funzioni definite all'interno del **setup**:

```
xTaskCreatePinnedToCore(sensorTask, "SensorTask", 4096, NULL, 1, &
    sensorTaskHandle, 0);
xTaskCreatePinnedToCore(mqttTask, "MQTTTask", 4096, NULL, 1, &
    mqttTaskHandle, 1);
xTaskCreatePinnedToCore(syncTimeTask, "SyncTimeTask", 4096, NULL, 1, &
    syncTimeTaskHandle, 1);
```

Listing 1: Creazione Task

che prendono come parametri:

- Puntatore alla funzione – il nome del task
- Descrizione del task
- Dimensione stack
- Parametri stack
- Priorità: possono essere assegnati valori da 0 a (`configMAX_PRIORITIES - 1`). Quest'ultimo viene definito all'interno del file di configurazione **FreeRTOSConfig.h**. Per task che hanno pari priorità, come nel nostro caso, viene assegnato valore 1. Valori bassi di priorità indicano attività a bassa priorità.
- Possibilità di accedere al task dall'esterno
- I valori 0 o 1 indicano il numero di indice della CPU a cui l'attività deve essere assegnata

Dopodiché vengono definiti i task creati, inserendo le istruzioni che dovranno essere svolte in loop.

Le istruzioni definite al loro interno, come per la lettura dei valori rilevati dai sensori, per la pubblicazione dei dati su MQTT e per il controllo delle soglie, verranno descritte in dettaglio successivamente.

All'interno del setup, prima della creazione dei task, come da codice sottostante, viene effettuata la connessione al WiFi, richiamando l'apposita funzione `connectToWiFi()` e vengono effettuati dei controlli iniziali per verificare la rilevazione e l'effettivo funzionamento e collegamento dei dispositivi utilizzati, compresa la sincronizzazione della data e dell'ora.

Per essere sicuri che la data e l'ora rilevata sia quella reale è stata inclusa la libreria `NTPClient` che consente la sincronizzazione della data e dell'ora da un server NTP.

Nel caso in cui non venga rilevato il sensore, viene visualizzato in output un messaggio di errore.

3 Sviluppo Progetto

```
void setup() {
  Serial.begin(115200);
  connectToWiFi();
  client.setServer(mqttBroker, mqttPort);

5   if (!bmp.begin()) {
      Serial.println("Impossibile trovare un sensore valido BMP280, controlla
      il cablaggio!");
      while (1);
    }

10  if (!lightSensor.begin()) {
      Serial.println("Impossibile trovare un sensore valido BH1750, controlla
      il cablaggio!");
      while (1);
    }

15  if (!rtc.begin()) {
      Serial.println("Impossibile trovare un sensore valido RTC, controlla il
      cablaggio!");
      while (1);
    }

20  syncTimeWithNTP();
}
```

Listing 2: Rilevazione sensori e sincronizzazione data e ora

Ogni sensore viene però prima inizializzato richiamando l'apposita libreria:

```
Adafruit_BMP280 bmp;
BH1750 lightSensor;
RTC_PCF8523 rtc;
```

Listing 3: Inizializzazione sensori

Viene definito un NTP Client per richiedere la data e l'ora ad un server NTP, così da avere sempre data e ora aggiornate. Sono stati specificati anche l'offset e l'intervallo di aggiornamento, espresso in millisecondi.

```
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 3600*2, 60000);
```

Listing 4: NTP Client

Viene poi inizializzato l'NTP Client all'interno della funzione per la connessione al WiFi `timeClient.begin()`.

Dopodiché nel setup viene richiamata la funzione apposita `syncTimeWithNTP()` per la sincronizzazione della data e dell'ora.

Nel codice sottostante, si può vedere in dettaglio come avviene la connessione al WiFi tramite `connectToWiFi()`.

La connessione alla rete si effettua attraverso la funzione `Wifi.begin(ssid, password)` alla quale viene passato l'SSID e la password del WiFi. Finché lo stato della connessione non risulta essere connesso, si ritenta la connessione.

```
void connectToWiFi() {
  Serial.println("Connessione al WiFi..");
  Wifi.begin(ssid, password);
}
```

```
5 while (WiFi.status() != WL_CONNECTED) {  
    delay(1000);  
    Serial.println("Connettendosi al WiFi...");  
}  
Serial.println("Connesso al WiFi");  
timeClient.begin();  
10 }
```

Listing 5: Connessione WiFi

Al fine di far comunicare i vari dispositivi, vengono utilizzate due librerie:

1. **WiFi:** per consentire la connessione alla rete e permette di definire server, client e inviare e ricevere messaggi.
2. **PubSubClient:** per scambiare messaggi MQTT.

Attraverso **WiFiClient** viene definito un nuovo client **espClient** che può connettersi alla rete e scambiare messaggi MQTT, attraverso PubSubClient `client(espClient)`.

Per semplificare la gestione del codice, vengono definite all’inizio le costanti e le variabili globali relative ai task, al WiFi, alla porta, all’indirizzo IP e ai topic MQTT.

```
TaskHandle_t sensorTaskHandle, mqttTaskHandle, syncTimeTaskHandle;  
QueueHandle_t dataQueue;  
WiFiClient espClient;  
PubSubClient client(espClient);  
5 const char *ssid = "TIM-63619464";  
const char *password = "uc4DfzHJHd36CZFJ";  
const char *mqttBroker = "192.168.1.12";  
const int mqttPort = 1883;  
  
10 // Costanti per i topic MQTT  
const char *temperatureTopic = "temperatura";  
const char *pressureTopic = "pressione";  
const char *lightTopic = "luminosit ";  
const char *dateTopic = "data";  
15 const char *timeTopic = "ora";
```

Listing 6: Dichiarazione costanti e variabili globali

3.4 ESP32, MQTT Broker e MQTT Client

L’acquisizione delle grandezze fisiche richieste avviene mediante i sensori collegati alla EPS32 che funge da nodo IoT. Questo microcontrollore, grazie alla connessione WiFi integrata e al protocollo MQTT, invierà i dati acquisiti al broker.

Come descritto precedentemente, il Raspberry assume il compito di MQTT Broker, mentre l’ESP32 funge da MQTT Client.

Il funzionamento di MQTT è il seguente:

- Il client MQTT stabilisce una connessione con il broker MQTT
- Una volta connesso, il client invia i messaggi al broker

3 Sviluppo Progetto

- Quando il broker riceve il messaggio, lo inoltra ai dispositivi interessati

Per fare ciò, prima di tutto, è stata inclusa la libreria **PubSubClient**, apposta per la messaggistica MQTT, che permette di ricevere e inviare messaggi.

All'interno del **setup** viene effettuata la connessione all'MQTT Broker, attraverso la funzione `client.setServer(mqttBroker, mqttPort)`, a cui viene passato l'indirizzo dell'MQTT Broker e la sua porta di default.

```
void setup() {  
  Serial.begin(115200);  
  connectToWiFi();  
  client.setServer(mqttBroker, mqttPort);  
}
```

Listing 7: Connessione MQTT Broker

Per la comunicazione e lo scambio dei dati, è necessario che la connessione con l'MQTT Broker sia sempre attiva, per questo è stato inserito all'interno del **void loop**, il controllo sullo stato della connessione.

Nel caso in cui, lo stato sia disconnesso, si tenta la connessione all'MQTT Broker, attraverso l'apposita funzione `client.connect()` ogni 5 secondi, visualizzando in output l'esito della connessione.

```
void loop() {  
  if (!client.connected()) {  
    connectToMQTT();  
  }  
  client.loop();  
  
  handleTelegramMessages(); // Controlla nuovi messaggi Telegram  
  
  delay(10000); // Attendi 10 secondi prima di controllare di nuovo  
}  
  
void connectToWiFi() {  
  Serial.println("Connessione al WiFi..");  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(1000);  
    Serial.println("Connettendosi al WiFi...");  
  }  
  Serial.println("Connesso al WiFi");  
  timeClient.begin();  
}  
  
void connectToMQTT() {  
  Serial.println("Connessione all MQTT");  
  while (!client.connected()) {  
    if (client.connect("ESP32Client")) {  
      Serial.println("Connesso all MQTT Broker");  
    } else {  
      Serial.print("Fallita la connessione all MQTT, rc=");  
      Serial.print(client.state());  
      Serial.println(" Riprova in 5 secondi");  
      delay(5000);  
    }  
  }  
}
```

```
}
```

Listing 8: Connessione WiFi e MQTT Broker

L'ESP32 si connette ad un server MQTT, avvia la connessione e attende di ricevere comandi da parte di un client. In questo caso i comandi arriveranno dal Raspberry Pi e dallo smartphone dell'utente iscritto al bot Telegram.

3.5 Sensori

L'ESP32 dovrà inviare costantemente i valori misurati dai sensori all'MQTT Broker, i quali verranno poi memorizzati all'interno di un database apposito. Nel caso in cui le misure rilevate superino una certa soglia, verrà generato e inviato un alert all'utente.

Per fare ciò è necessario sottoscrivere il client MQTT ai topic relativi alle misure rilevate dai vari sensori, richiamando nel **setup** la funzione `subscribeToTopics()` così da permettere la verifica della corrispondenza tra i topic caricati e quelli richiesti dall'utente.

```
void subscribeToTopics() {  
    if (client.connected()) {  
        client.subscribe(temperatureTopic);  
        client.subscribe(pressureTopic);  
5      client.subscribe(lightTopic);  
        client.subscribe(dateTopic);  
        client.subscribe(timeTopic);  
    }  
}
```

Listing 9: Sottoscrizione topic

Ora, vediamo in dettaglio come sono stati sviluppati i vari task.

3.5.1 Sensor Task

Il BMP280 è un sensore utilizzato per misurare temperatura e pressione dell'ambiente circostante, mentre il BH1750 è un sensore in grado di rilevare la luminosità dell'ambiente.

Per poter gestire la lettura e la memorizzazione dei dati dai sensori, viene definita una struct con tre variabili float rispettivamente per temperatura, pressione e luminosità e due array per memorizzare data e ora.

```
struct SensorData {  
    float temperature;  
    float pressure;  
4    float light;  
    char date[11];  
    char time[9];  
};
```

Listing 10: Struct SensorData

La rilevazione dei dati dai sensori avviene mediante un unico task **sensorTask()** e deve essere costante e ciclica, per questo sono state inserite le apposite funzioni di lettura all'interno di un ciclo `while()`. La lettura dei dati avviene mediante le apposite funzioni:

- `bmp.readTemperature()`: per la temperatura
- `bmp.readPressure() / 100.0`: per la pressione
- `lightSensor.readLightLevel()`: per la luminosità
- `rtc.now()`: per la data e ora, che vengono poi divise in anno, mese e giorno e ora, minuti e secondi.

```
void sensorTask(void *parameter) {  
    (void)parameter;  
3    TickType_t xLastWakeTime = xTaskGetTickCount();  
  
    while (1) {  
        // Leggi dati dai sensori  
        SensorData data;  
8        data.temperature = bmp.readTemperature();  
        data.pressure = bmp.readPressure() / 100.0; // Pressione in hPa  
        data.light = lightSensor.readLightLevel();  
  
        // Leggi l'ora corrente dal RTC  
13        DateTime now = rtc.now();  
        snprintf(data.date, sizeof(data.date), "%04d/%02d/%02d", now.year(), now  
            .month(), now.day());  
        snprintf(data.time, sizeof(data.time), "%02d:%02d:%02d", now.hour(), now  
            .minute(), now.second());
```

Listing 11: Task Lettura dati

Per poter inviare i dati all'MQTT Broker viene creata una coda

```
dataQueue = xQueueCreate(10, sizeof(SensorData));
```

in grado di contenere 10 puntatori alla struttura `SensorData`.

In particolare, una volta rilevati i dati dai sensori e memorizzati all'interno della struct, i dati vengono inviati alla coda tramite la funzione `xQueueSend()`.

```
    if (xQueueSend(dataQueue, &data, portMAX_DELAY) != pdPASS) {  
        Serial.println("Coda piena, impossibile inviare i dati dei sensori");  
    }  
  
5    vTaskDelayUntil(&xLastWakeTime, 10000 / portTICK_PERIOD_MS); // Ritardo  
    di 10 secondi  
    }  
}
```

Listing 12: Invio dati alla coda

3.5.2 MQTT Task

Il secondo task implementato è quello relativo all'MQTT, il quale ciclicamente riceve i dati dalla coda tramite la funzione `xQueueReceive()` e crea una stringa formattata con i dati dei sensori separati dalle virgole. Dopodiché l'ESP32 si occupa dell'invio dei dati all'MQTT Broker, visualizzando in output l'esito.

3 Sviluppo Progetto

```
while (1) {
    // Ricevi i dati dalla coda
    if (xQueueReceive(dataQueue, &data, portMAX_DELAY) == pdPASS) {
        // Crea una stringa formattata con i dati dei sensori separati da
        virgole
5      char payload[100];
        snprintf(payload, sizeof(payload), "%.2f,%.2f,%.2f,%s,%s", data.
        temperature, data.pressure, data.light, data.date, data.time);

        // Pubblica i dati su MQTT
10      if (client.publish("sensori", payload)) {
        Serial.print("Dati inviati al Broker MQTT: ");
        Serial.println(payload);
      } else {
        Serial.println("Fallito nell'inviare i dati al Broker MQTT");
      }
    }
}
```

Listing 13: Ricezione e pubblicazione dati

Vengono definite delle variabili e costanti per il controllo del superamento della soglia, tenendo conto dell'isteresi del sensore. Per ogni sensore è stato definito:

- **Valore di soglia superiore:** valore di soglia massimo + isteresi
- **Valore di soglia inferiore:** valore di soglia minimo + isteresi

```
bool temperatureThresholdEnabled = true; // Variabile per tenere traccia
    dello stato del monitoraggio della soglia
2 const float temperatureThresholdHigh = 5.5; // Soglia alta della temperatura
    in C
const float temperatureThresholdLow = 4.5; // Soglia bassa della temperatura
    in C

bool lightThresholdEnabled = true; // Variabile per tenere traccia dello
    stato del monitoraggio della soglia
const float lightThresholdHigh = 530.0; // Soglia alta della luminosit in
    Lumen
7 const float lightThresholdLow = 450.0; // Soglia bassa della luminosit in
    Lumen

bool pressureThresholdEnabled = true; // Variabile per tenere traccia dello
    stato del monitoraggio della soglia
const float pressureThresholdHigh = 1022.0; // Soglia alta della pressione
    in hPa
const float pressureThresholdLow = 1015.0; // Soglia bassa della pressione
    in hPa
```

Listing 14: Variabili e costanti per monitoraggio soglia

Nel caso in cui il monitoraggio della soglia sia abilitato

```
2 bool temperatureThresholdEnabled = true;
bool pressureThresholdEnabled = true;
bool lightThresholdEnabled = true;
```

si controlla se il valore inviato è maggiore della soglia superiore o minore della soglia inferiore; in entrambi i casi, verrà inviato un alert al bot Telegram comunicando che la grandezza misurata

3 Sviluppo Progetto

(temperatura/pressione/luminosità) è troppo alta o troppo bassa e si disattiva il monitoraggio, impostando a **false** la variabile booleana, così da evitare che arrivino all'utente continui messaggi.

Il monitoraggio viene riattivato solo quando i valori misurati tornano ad essere compresi tra i due valori di soglia impostati.

```
void mqttTask(void *parameter) {
2   (void)parameter;
   SensorData data;

   while (1) {
       // Ricevi i dati dalla coda
7       if (xQueueReceive(dataQueue, &data, portMAX_DELAY) == pdPASS) {
           // Crea una stringa formattata con i dati dei sensori separati da virgole
           char payload[100];
           snprintf(payload, sizeof(payload), "%.2f,%.2f,%.2f,%s,%s", data.
temperature, data.pressure, data.light, data.date, data.time);

12          // Pubblica i dati su MQTT
           if (client.publish("sensori", payload)) {
               Serial.print("Dati inviati al Broker MQTT: ");
               Serial.println(payload);
           } else {
17              Serial.println("Fallito nell'inviare i dati al Broker MQTT");
           }

22          // Controllo della soglia solo se il monitoraggio della soglia
          abilitato
           if (temperatureThresholdEnabled && data.temperature >
temperatureThresholdHigh) {
               sendTelegramMessage("Attenzione: Temperatura troppo alta[C]!",
String(data.temperature).c_str());
               temperatureThresholdEnabled = false; // Disattiva il monitoraggio
della soglia
27              } else if (temperatureThresholdEnabled && data.temperature <
temperatureThresholdLow) {
               sendTelegramMessage("Attenzione: Temperatura troppo bassa [C]!",
String(data.temperature).c_str());
               temperatureThresholdEnabled = false; // Disattiva il monitoraggio
della soglia
           }
           // Riattiva il monitoraggio della soglia se la temperatura scende
sotto la soglia
32           if (!temperatureThresholdEnabled && data.temperature <=
temperatureThresholdHigh) {
               temperatureThresholdEnabled = true;
           }

           // Controllo della soglia solo se il monitoraggio della soglia
37          abilitato
           if (pressureThresholdEnabled && data.pressure > pressureThresholdHigh
) {
               sendTelegramMessage("Attenzione: Pressione troppo alta[Pa]!",
```

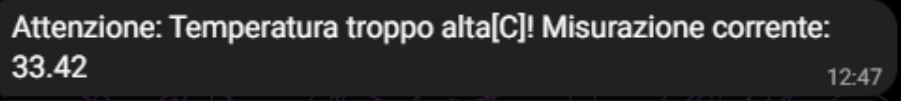
```
String(data.pressure).c_str());
    pressureThresholdEnabled = false; // Disattiva il monitoraggio
della soglia
    }else if(pressureThresholdEnabled&& data.pressure <
42 pressureThresholdLow){
    sendTelegramMessage("Attenzione: Pressione troppo bassa[Pa]!",
String(data.pressure).c_str());
    pressureThresholdEnabled = false; // Disattiva il monitoraggio
della soglia
    }
    // Riattiva il monitoraggio della soglia se la pressione scende sotto
la soglia
    if (!pressureThresholdEnabled && data.pressure <=
47 pressureThresholdHigh) {
    pressureThresholdEnabled = true;

    }

52

    // Controlla le soglie di luminosità
    if (lightThresholdEnabled) {
        if (data.light > lightThresholdHigh) {
            sendTelegramMessage("Attenzione: Luminosità troppo alta!", String
(data.light).c_str());
57            lightThresholdEnabled=false;
        } else if (data.light < lightThresholdLow) {
            sendTelegramMessage("Attenzione: Luminosità troppo bassa!",
String(data.light).c_str());
            lightThresholdEnabled=false;
        } else if (data.light >= lightThresholdLow && data.light <=
62 lightThresholdHigh) {
            lightThresholdEnabled = true; // Reset alert quando la luminosità
rientra nei limiti
        }
    }
}
}
67 }
```

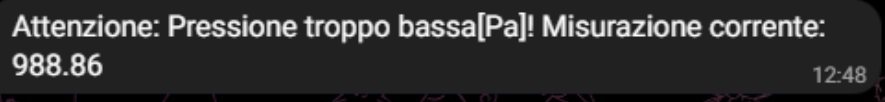
Listing 15: MQTT Task



Attenzione: Temperatura troppo alta[C]! Misurazione corrente:
33.42

12:47

Figura 11: Alert superamento soglia temperatura



Attenzione: Pressione troppo bassa[Pa]! Misurazione corrente:
988.86

12:48

Figura 12: Alert superamento soglia pressione

3.5.3 SyncTimeTask

L'ultimo task creato viene impiegato per mantenere la data e l'ora dell'RTC aggiornata.

Ogni ora viene prevista una sincronizzazione richiamando la funzione `syncTimeWithNTP()` che, dopo aver richiesto la data e l'ora al server NTP, le sovrascrive all'interno dell'RTC. Per fare ciò vengono impiegate due funzioni:

- `getEpochTime()`: viene restituisce il tempo trascorso dall'anno in cui fu introdotto il GMT
- `rtc.adjust(DateTime(epochTime))`: sovrascrive la data dell'RTC con data e ora rilevata dal server NTP.

```
void syncTimeTask(void *parameter) {  
    (void)parameter;  
    while (1) {  
        syncTimeWithNTP();  
5        vTaskDelay(3600000 / portTICK_PERIOD_MS); // Sincronizza ogni ora  
    }  
}
```

Listing 16: Task sincronizzazione data e ora

```
void syncTimeWithNTP() {  
    while (!timeClient.update()) {  
3        timeClient.forceUpdate();  
    }  
  
    unsigned long epochTime = timeClient.getEpochTime();  
    rtc.adjust(DateTime(epochTime));  
8 }
```

Listing 17: Sincronizzazione NTP

3.6 Bot Telegram

Telegram è un servizio di messaggistica che permette la creazione di bot. Di fatti, il bot Telegram permette di far interagire il client attraverso messaggi Telegram, in particolare permetterà all'utente di visualizzare i valori misurati, gli eventuali alert ed i grafici sugli andamenti delle grandezze, in base ad un particolare range.

Per implementare un nuovo bot, viene utilizzato BotFather, che è il bot padre da cui si possono creare altri bot. Si cerca su Telegram **@BotFather** e si invia sulla chat il comando `/newbot`. Dopodiché il BotFather chiederà il nome e l'identificativo da associare al nuovo bot, per poi fornire il token identificativo, da salvare perché necessario per poter utilizzare le API Telegram e per la programmazione del bot. Più utenti possono iscriversi al Bot Telegram, inserendo il nickname della chat. Ogni utente ha un proprio chat ID, che verrà memorizzato all'interno del vettore `chatIDs`.

```
const char *telegramBotToken = "6728633709:  
    AAFXKIkfqvrAS2ub1CwPKIJ5PIdrKqdgEps";  
std::vector<String> chatIDs; // Vettore per memorizzare gli ID chat
```

Listing 18: Codice identificativo bot Telegram

3 Sviluppo Progetto

Una volta generato il bot, si passa all'implementazione del codice. Nel nostro caso, il bot viene configurato sia per l'invio che per la ricezione dei messaggi.

Per consentire la comunicazione tra i dispositivi è stata inclusa la libreria **HTTPClient**, che permette di creare con facilità richieste HTTP a un server web. Con l'ausilio di questa libreria, viene implementata la funzione `sendTelegramMessage()` per l'invio dei messaggi.

L'ESP32 invia una richiesta HTTP alle API di Telegram, di cui abbiamo specificato l'url, le quali restituiscono una risposta al client (ESP32).

Per prima cosa si verifica se ci sono chat ID disponibili e si controlla se ci sono nuovi messaggi per ottenere i chat ID, tramite la funzione `handleTelegramMessages()`.

Quando è presente almeno un chat ID, per instaurare la comunicazione viene utilizzata la funzione `http.begin(url)` dove l'**url** è stato ottenuto combinando la parte base dell'URL delle API di Telegram, il token del bot Telegram, il chat ID ed il messaggio inviato dall'utente e tramite il **GET()** viene inviata la richiesta di dati di una risorsa specifica. Se la richiesta restituisce valore positivo, viene visualizzata la conferma dell'invio, altrimenti viene mostrato il messaggio di errore.

```
void sendTelegramMessage(const char *message, const char *measurement) {
    // Attendi finch non ci sono chat ID disponibili
    while (chatIDs.empty()) {
        Serial.println("Nessun chat ID disponibile. In attesa di ottenere i chat
ID...");
5      handleTelegramMessages(); // Controlla i nuovi messaggi per ottenere i
chat ID
        delay(5000); // Attendi 5 secondi prima di controllare di nuovo
    }

    if (WiFi.status() == WL_CONNECTED && !chatIDs.empty()) { // Verifica che
chatIDs non sia vuoto
10      HTTPClient http;
        for (const String& chatID : chatIDs) {
            String url = String("https://api.telegram.org/bot") + telegramBotToken
+ "/sendMessage?chat_id=" + chatID + "&text=" + message + " Misurazione
corrente: " + measurement;
            http.begin(url);
            int httpCode = http.GET();
15          if (httpCode > 0) {
                Serial.println("Messaggio Telegram inviato correttamente a chatID: "
+ chatID);
            } else {
                Serial.println("Errore nell'inviare il messaggio Telegram a chatID:
" + chatID);
            }
20          http.end();
        }
    } else {
        Serial.println("Errore: chatIDs vuoto o WiFi non connesso.");
    }
25 }
```

Listing 19: Funzione invio messaggi Telegram

Per la gestione dei messaggi in arrivo di Telegram è stata sviluppata la funzione `handleTelegramMessages()`.

3 Sviluppo Progetto

Prima di tutto si definisce un URL combinando la parte base dell'URL delle API di Telegram, il token del bot Telegram ed il percorso `/getUpdates`. Dopo aver avviato la connessione HTTP, viene inviata una richiesta con il **GET()** e se la risposta è positiva, viene memorizzata all'interno della stringa **payload**.

Dopodiché si crea:

- **Documento JSON:** in cui verranno memorizzate le risposte alle richieste effettuate, dopo averle analizzate.

```
deserializeJson(doc, payload);
```

- **Vettore JSON:** utilizzato per convertire l'array **result** dalla risposta JSON.

```
JsonArray updates = doc["result"].as<JsonArray>();
```

Per ogni oggetto appartenente al vettore sopra definito, viene estratto l'ID della chat che verrà aggiunto, se non presente, all'elenco degli ID.

```
void handleTelegramMessages() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = String("https://api.telegram.org/bot") + telegramBotToken +
    "/getUpdates";
    http.begin(url);
    int httpCode = http.GET();
    if (httpCode > 0) {
      String payload = http.getString();
      DynamicJsonDocument doc(1024);
      DeserializationError error = deserializeJson(doc, payload);
      if (!error) {
        for (JsonObject update : doc["result"].as<JsonArray>()) {
          String chat_id = update["message"]["chat"]["id"].as<String>();
          if (std::find(chatIDs.begin(), chatIDs.end(), chat_id) == chatIDs.
end()) {
            chatIDs.push_back(chat_id); // Aggiungi il chat_id se non
gi presente
            Serial.print("Nuovo chat ID aggiunto: ");
            Serial.println(chat_id);
          }
        }
      } else {
        Serial.println("Errore nella deserializzazione del payload JSON");
      }
    } else {
      Serial.println("Errore HTTP nella richiesta getUpdates");
    }
    http.end();
  }
}
```

Listing 20: Funzione ricezione messaggi Telegram e aggiunta chat ID

L'utente potrà inviare sulla chat i seguenti comandi:

1. **/start:** per avviare la comunicazione ed il bot invierà una risposta preimpostata da noi.

2. **/leggi:** per leggere i dati memorizzati all'interno del database relativi ad un range di tempo.

Entrambe le funzionalità verranno spiegate nel paragrafo successivo.

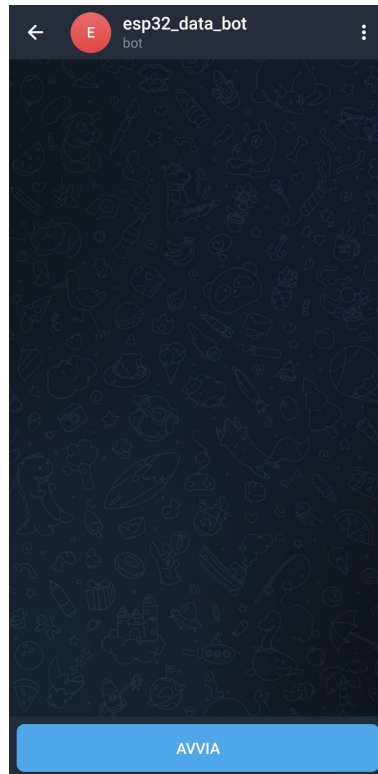


Figura 13: Bot Telegram

3.7 Creazione Database e caricamento dati

Il database utilizzato per questo progetto è un database **MySQL**, gestito con **phpMyAdmin** e viene eseguito su una macchina Windows locale ed un **server Apache**. Per poter configurare e gestire facilmente il DB, è stato installato **XAMPP** e creato il DB tramite l'interfaccia utente di phpMyAdmin.

XAMPP è un software che permette di testare il database in locale, phpMyAdmin è un software che permette la gestione di MySQL.

Nella schermata principale di XAMPP vengono mostrati i vari servizi disponibili; nel nostro caso vengono selezionati Apache e MySQL. Cliccando su **Admin** in corrispondenza di MySQL, si aprirà la pagina di **phpMyAdmin**.

Per creare il database, viene effettuato il login con l'account root di default e poi si clicca su **create database** e a seguire su **create table** per generare la tabella **sensor data** che conterrà tutti i parametri dei sensori. Gli attributi della tabella sono stati scelti in maniera tale da avere degli attributi di tipo:

- **float:** per pressione e temperatura
- **int autoincrementale:** per gli ID

3 Sviluppo Progetto

- **DATETIME:** per data e ora del sensore RTC

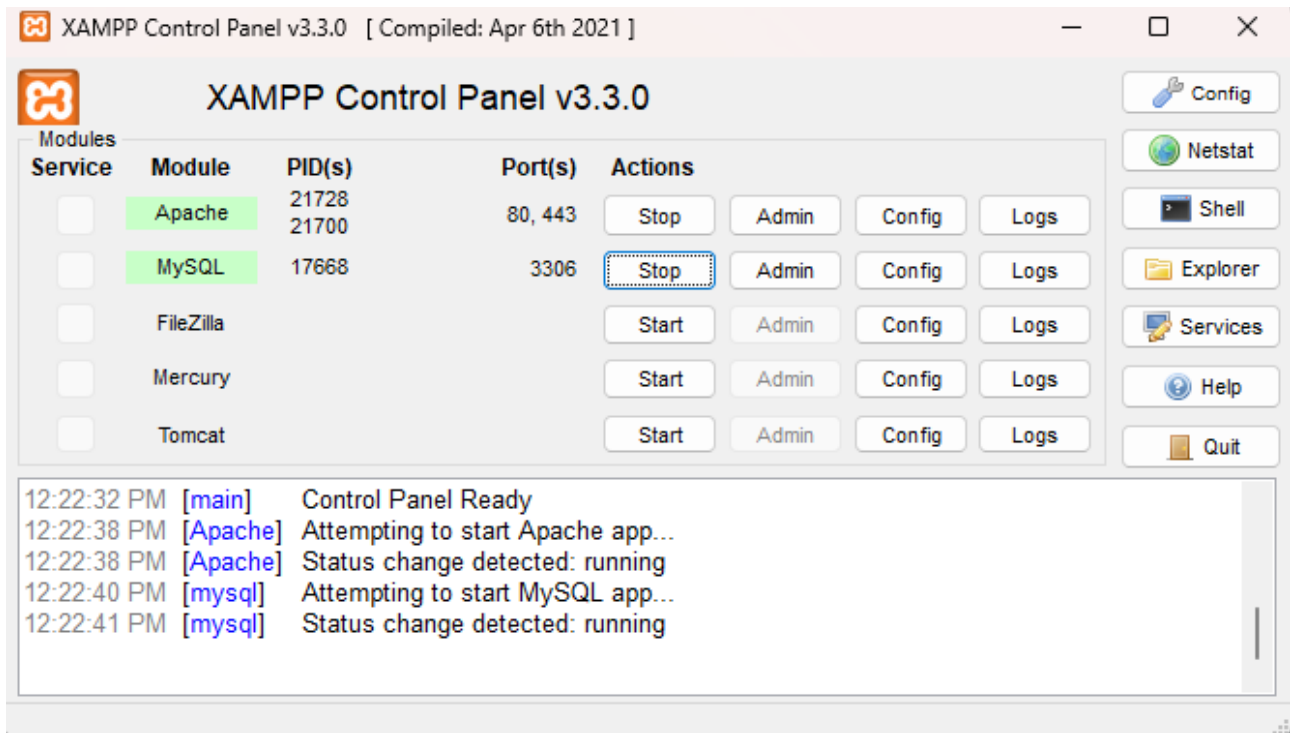


Figura 14: GUI Xampp

Una volta creato il database, si effettua la connessione ad esso, tramite apposita funzione **mysql.connector.connect** a cui vengono passati:

- L'indirizzo del server MySQL
- Il nome utente associato al database
- La password del database
- Il nome del database

```
broker_address = '192.168.1.12' # Indirizzo del broker MQTT
db_config = {
    'host': '192.168.1.13', # Indirizzo del server MySQL
    'user': 'brugia', # Nome utente del database MySQL
    'password': 'halo3000', # Password del database MySQL
    'database': 'esp32_data' # Nome del database MySQL
}
```

Listing 21: Dati configurazione database

```
conn = mysql.connector.connect(**db_config)
cursor = conn.cursor()
3 print("Connessione al database MySQL avvenuta con successo")
```

Listing 22: Connessione al database

3 Sviluppo Progetto

Per poter caricare i dati rilevati all'interno del database è necessario utilizzare una funzione apposita per sottoscrivere un client MQTT ai topic necessari. Prima di tutto, con la funzione `sniff_mqtt_messages()` si crea un nuovo client MQTT, si imposta la funzione `on_message()` come callback per gestire i messaggi MQTT ricevuti, si connette il client al broker, lo si iscrive ai topics definiti all'interno dell'array

```
topics = ["sensori"]
```

e si avvia il loop di rete per ricevere i messaggi.

```
def sniff_mqtt_messages(broker_address, topics):
    client = mqtt.Client(callback_api_version=2)
    client.on_message = on_message

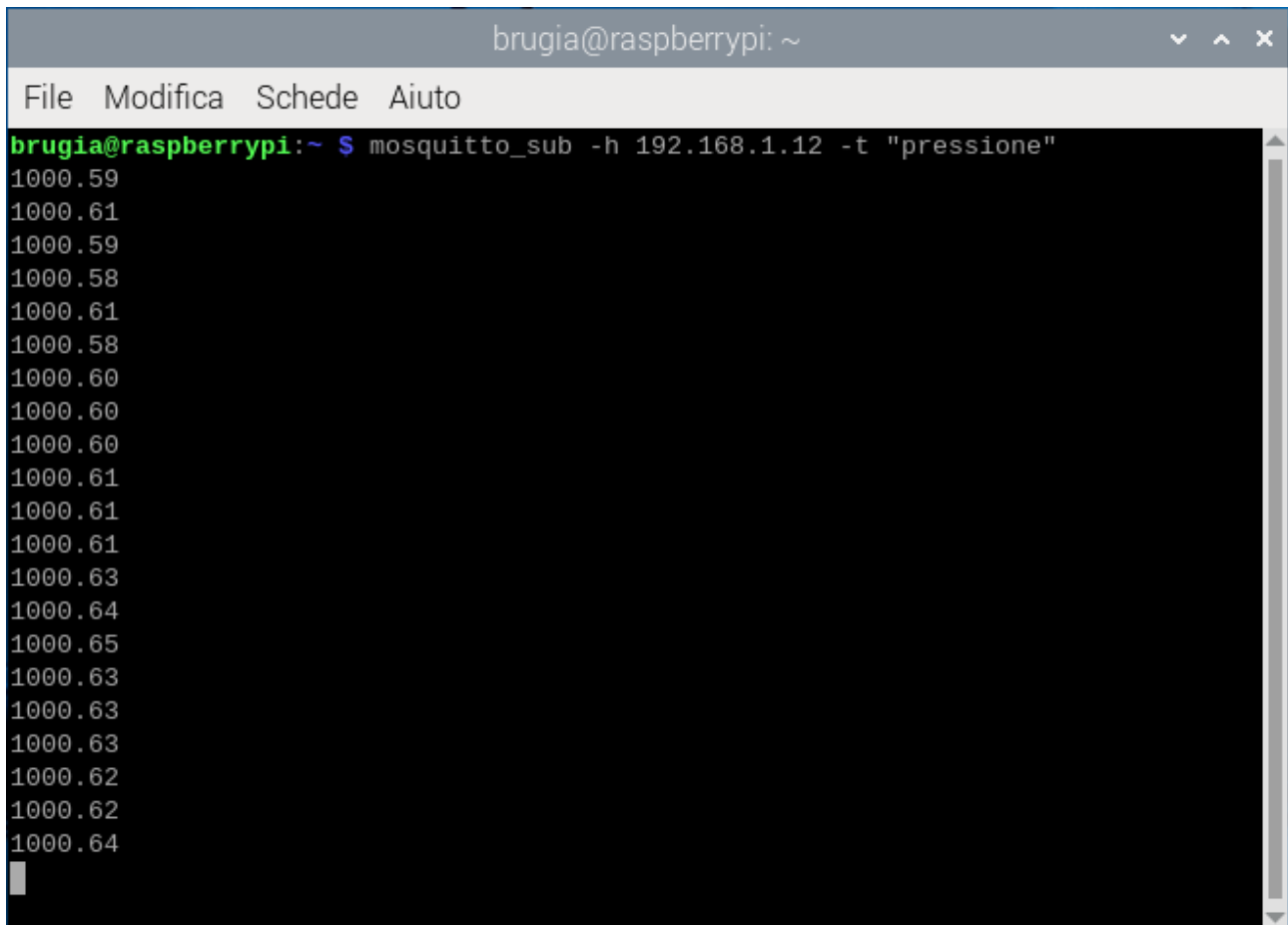
    # Connessione al broker MQTT
    client.connect(broker_address)
    print("Connessione al broker MQTT avvenuta con successo")

    # Sottoscrizione ai topic
    for topic in topics:
        client.subscribe(topic)
        print(f"Sottoscritto al topic: {topic}")

    # Avvia il loop di rete del client MQTT per ricevere i messaggi
    client.loop_forever()
```

Listing 23: Funzione per configurare e avviare client MQTT

Nel dettaglio, la funzione `on_message` decodifica il payload del messaggio separato dalle virgole e memorizza i primi 3 valori all'interno di tre variabili float, rispettivamente **temperatura**, **pressione** e **luminosità** ed i restanti due all'interno di due array rispettivamente **data_{value}**



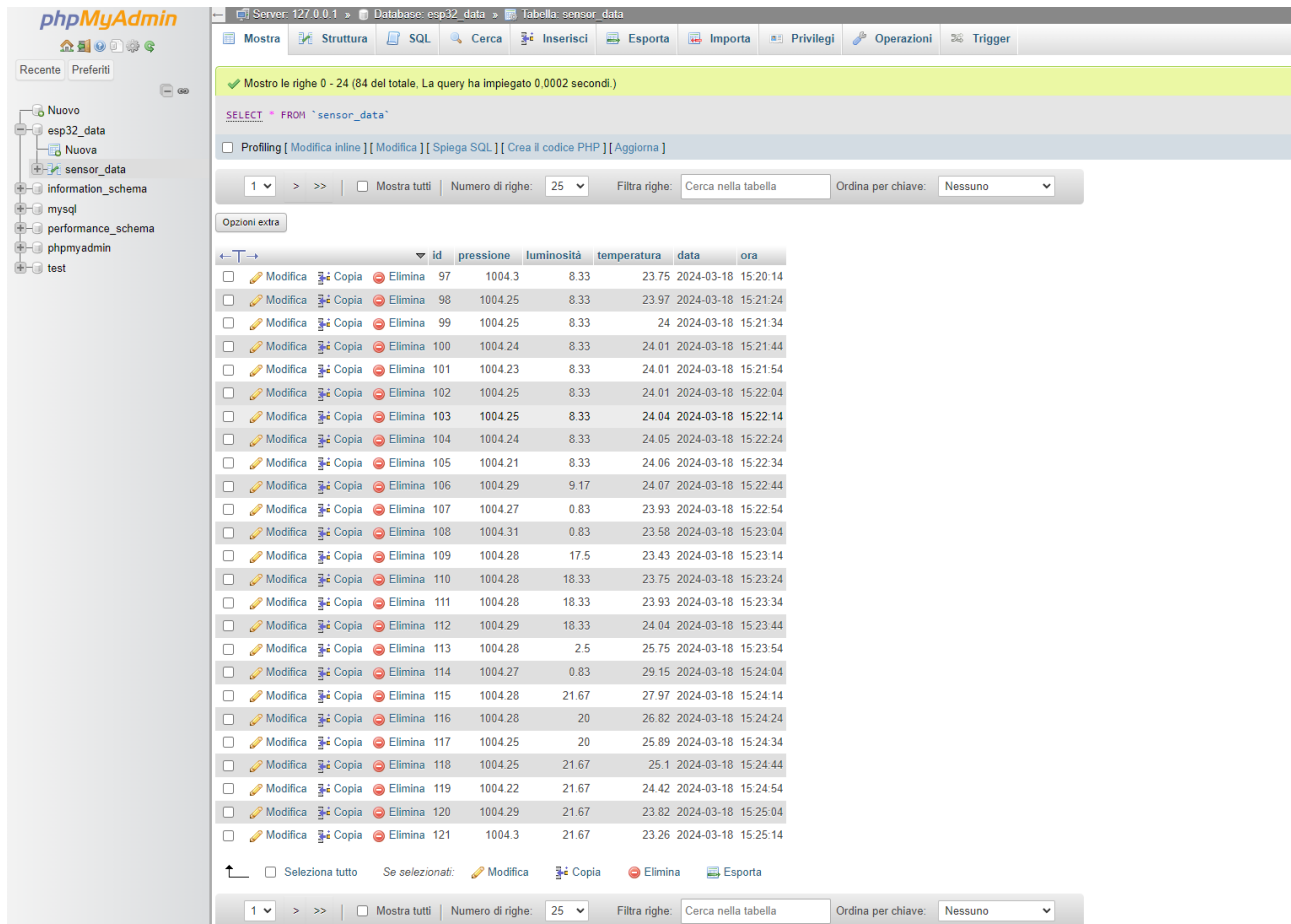
A terminal window titled 'brugia@raspberrypi: ~' with a menu bar containing 'File', 'Modifica', 'Schede', and 'Aiuto'. The terminal shows the command 'mosquitto_sub -h 192.168.1.12 -t "pressione"' being executed. Below the command, a list of 20 floating-point numbers representing pressure values is displayed, ranging from 1000.58 to 1000.65. The values are: 1000.59, 1000.61, 1000.59, 1000.58, 1000.61, 1000.58, 1000.60, 1000.60, 1000.60, 1000.61, 1000.61, 1000.61, 1000.63, 1000.64, 1000.65, 1000.63, 1000.63, 1000.63, 1000.62, 1000.62, and 1000.64. A vertical scrollbar is visible on the right side of the terminal window.

```
brugia@raspberrypi:~ $ mosquitto_sub -h 192.168.1.12 -t "pressione"
1000.59
1000.61
1000.59
1000.58
1000.61
1000.58
1000.60
1000.60
1000.60
1000.61
1000.61
1000.61
1000.63
1000.64
1000.65
1000.63
1000.63
1000.63
1000.62
1000.62
1000.64
```

Figura 15: Pressione inviata al broker

Nell'immagine sottostante possiamo vedere il database creato con i valori caricati.

3 Sviluppo Progetto



Server: 127.0.0.1 Database: esp32_data Tabella: sensor_data

Mostra Struttura SQL Cerca Inserisci Esporta Importa Privilegi Operazioni Trigger

Mostrare le righe 0 - 24 (84 del totale, La query ha impiegato 0.0002 secondi.)

`SELECT * FROM `sensor_data``

Profiling [Modifica inline] [Modifica] [Spiega SQL] [Crea il codice PHP] [Aggiorna]

1 > >> ☐ Mostra tutti Numero di righe: 25 Filtra righe: Cerca nella tabella Ordina per chiave: Nessuno

Opzioni extra

				id	pressione	luminosità	temperatura	data	ora
☐	Modifica	Copia	Elimina	97	1004.3	8.33	23.75	2024-03-18	15:20:14
☐	Modifica	Copia	Elimina	98	1004.25	8.33	23.97	2024-03-18	15:21:24
☐	Modifica	Copia	Elimina	99	1004.25	8.33	24	2024-03-18	15:21:34
☐	Modifica	Copia	Elimina	100	1004.24	8.33	24.01	2024-03-18	15:21:44
☐	Modifica	Copia	Elimina	101	1004.23	8.33	24.01	2024-03-18	15:21:54
☐	Modifica	Copia	Elimina	102	1004.25	8.33	24.01	2024-03-18	15:22:04
☐	Modifica	Copia	Elimina	103	1004.25	8.33	24.04	2024-03-18	15:22:14
☐	Modifica	Copia	Elimina	104	1004.24	8.33	24.05	2024-03-18	15:22:24
☐	Modifica	Copia	Elimina	105	1004.21	8.33	24.06	2024-03-18	15:22:34
☐	Modifica	Copia	Elimina	106	1004.29	9.17	24.07	2024-03-18	15:22:44
☐	Modifica	Copia	Elimina	107	1004.27	0.83	23.93	2024-03-18	15:22:54
☐	Modifica	Copia	Elimina	108	1004.31	0.83	23.58	2024-03-18	15:23:04
☐	Modifica	Copia	Elimina	109	1004.28	17.5	23.43	2024-03-18	15:23:14
☐	Modifica	Copia	Elimina	110	1004.28	18.33	23.75	2024-03-18	15:23:24
☐	Modifica	Copia	Elimina	111	1004.28	18.33	23.93	2024-03-18	15:23:34
☐	Modifica	Copia	Elimina	112	1004.29	18.33	24.04	2024-03-18	15:23:44
☐	Modifica	Copia	Elimina	113	1004.28	2.5	25.75	2024-03-18	15:23:54
☐	Modifica	Copia	Elimina	114	1004.27	0.83	29.15	2024-03-18	15:24:04
☐	Modifica	Copia	Elimina	115	1004.28	21.67	27.97	2024-03-18	15:24:14
☐	Modifica	Copia	Elimina	116	1004.28	20	26.82	2024-03-18	15:24:24
☐	Modifica	Copia	Elimina	117	1004.25	20	25.89	2024-03-18	15:24:34
☐	Modifica	Copia	Elimina	118	1004.25	21.67	25.1	2024-03-18	15:24:44
☐	Modifica	Copia	Elimina	119	1004.22	21.67	24.42	2024-03-18	15:24:54
☐	Modifica	Copia	Elimina	120	1004.29	21.67	23.82	2024-03-18	15:25:04
☐	Modifica	Copia	Elimina	121	1004.3	21.67	23.26	2024-03-18	15:25:14

1 > >> ☐ Mostra tutti Numero di righe: 25 Filtra righe: Cerca nella tabella Ordina per chiave: Nessuno

Figura 16: Database

Inviando sulla chat Telegram il comando `/start`, verrà visualizzato il messaggio di benvenuto con la richiesta di inserimento informazioni per ottenere i dati.

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    """Send a message when the command /start is issued."""
    user = update.effective_user
    await update.message.reply_html(
        rf"Ciao {user.mention_html()}, inserisci read+data+topic per
        ottenere i dati!",
        reply_markup=ForceReply(selective=True),
    )
```

Listing 24: Comando start

Inviando nella chat il comando `/leggi`, seguita da data, ora e topic, l'utente otterrà come risposta l'elenco dei dati memorizzati nel database o nel caso in cui il formato inserito non sia valido, verrà inviato all'utente un messaggio di errore comunicando il formato corretto da utilizzare per la richiesta.

Per estrarre i dati dal database viene effettuata un'interrogazione di esso mediante una query in cui si vanno a selezionare dalla tabella **sensor_data** i valori relativi alla data e all'ora richiesta dall'utente, attraverso la funzione `cursor.fetchall()`.

Se esiste una corrispondenza dei dati con la data e l'ora richieste, verranno inviati al bot Telegram rispettivamente i valori di pressione, luminosità e temperatura. Altrimenti, verrà

3 Sviluppo Progetto

generato un messaggio di errore o verrà comunicato che non esiste alcun dato nella data e ora richieste.

```
async def leggi(update, context):
    user = update.effective_user

    if len(context.args) < 2:
        await context.bot.send_message(chat_id=user.id, text="Devi
5      specificare una data e un'ora nel formato 'YYYY-MM-DD HH:MM:SS' e un
        topic.")
        return

    print("Valore di context.args[0]:", context.args[0])
    print("Valore di context.args[1]:", context.args[1])
10

    try:
        datetime_str = context.args[0] + ' ' + context.args[1]
        datetime_filter = datetime.strptime(datetime_str, '%Y-%m-%d %H:%M:%S
        ')

        query = "SELECT * FROM sensor_data WHERE DATE(data) >= %s AND TIME(
15      ora) >= %s"
        cursor.execute(query, (datetime_filter.date(), datetime_filter.time
        ()))
        results = cursor.fetchall()

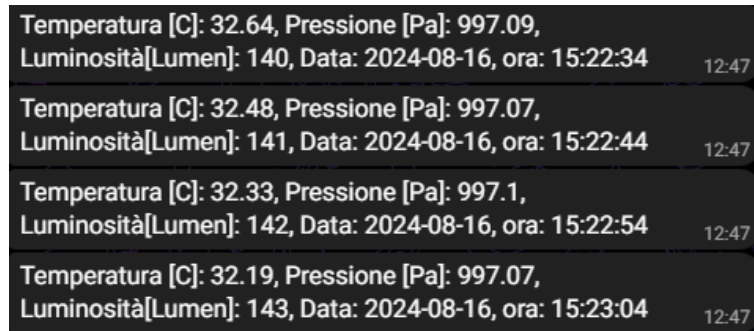
        if results:
20

            timestamps = [datetime.strptime(row[4].strftime('%Y-%m-%d') + '
            ' + (datetime.min + row[5]).strftime('%H:%M:%S'), '%Y-%m-%d %H:%M:%S')
            for row in results]
            pressione = [row[2] for row in results]
            luminosita = [row[3] for row in results]
25          temperatura = [row[1] for row in results]
```

Listing 25: Comando leggi con estrazione dati

```
1      if results:
        for row in results:
            message = f"Temperatura [C]: {row[1]}, Pressione [Pa]: {
            row[2]}, Luminosit [Lumen]: {row[0]}, Data: {row[4]}, ora: {row[5]}"
            await context.bot.send_message(chat_id=user.id, text=
            message)
        else:
6      await context.bot.send_message(chat_id=user.id, text="Nessun
        valore trovato per la data e l'ora specificate.")
    except ValueError:
        await context.bot.send_message(chat_id=user.id, text="Formato data e
        ora non valido. Utilizza 'YYYY-MM-DD HH:MM:SS'.")
```

Listing 26: Comando leggi con estrazione dati



Temperatura [C]: 32.64, Pressione [Pa]: 997.09, Luminosità[Lumen]: 140, Data: 2024-08-16, ora: 15:22:34	12:47
Temperatura [C]: 32.48, Pressione [Pa]: 997.07, Luminosità[Lumen]: 141, Data: 2024-08-16, ora: 15:22:44	12:47
Temperatura [C]: 32.33, Pressione [Pa]: 997.1, Luminosità[Lumen]: 142, Data: 2024-08-16, ora: 15:22:54	12:47
Temperatura [C]: 32.19, Pressione [Pa]: 997.07, Luminosità[Lumen]: 143, Data: 2024-08-16, ora: 15:23:04	12:47

Figura 17: Comando Bot "leggi"



Figura 18: Messaggio di errore

3.8 Grafici

L'utente può richiedere la generazione dei grafici a partire da una determinata data ed ora.

Viene effettuata l'estrazione dei dati dal database per ogni topic, come fatto in precedenza per il comando **leggi** e si calcola per ogni misura il valore massimo, il valore minimo e la media dei valori, tramite le apposite funzioni **max()**, **min()** e **mean()**, del periodo selezionato.

Per ogni topic verrà realizzato un grafico, contenente i valori memorizzati nel database a partire dalla data e ora richiesta dall'utente, mostrando anche le statistiche calcolate.

Una volta generato il grafico, viene salvato come immagine, inviato al bot Telegram tramite l'apposita funzione `context.bot.send_photo()` e viene poi eliminato il file.

```
5     pressione_media = np.mean(pressione)
        pressione_minimo = np.min(pressione)
        pressione_massimo = np.max(pressione)

        luminosita_media = np.mean(luminosita)
        luminosita_minimo = np.min(luminosita)
        luminosita_massimo = np.max(luminosita)

        temperatura_media = np.mean(temperatura)
        temperatura_minimo = np.min(temperatura)
        temperatura_massimo = np.max(temperatura)

10     # Funzione per creare il grafico e salvarlo
        def crea_grafico(timestamps, valori, parametro, media, minimo,
            massimo, nome_file):
15         plt.figure(figsize=(10, 6))
            plt.plot(timestamps, valori, label=parametro, marker='o')
            plt.axhline(y=media, color='blue', linestyle='--', label=f'
Media: {media:.2f}')
            plt.axhline(y=minimo, color='green', linestyle='--', label=f'
'Minimo: {minimo:.2f}')
            plt.axhline(y=massimo, color='red', linestyle='--', label=f'
Massimo: {massimo:.2f}')
```

```
20         plt.legend(loc='upper left')
           plt.xlabel('Data e Ora')
           plt.ylabel('Valore')
           plt.title(f'{parametro} nel Tempo')

25         # Aggiungi le statistiche sul lato del grafico
           plt.annotate(f'Media: {media:.2f}\nMinimo: {minimo:.2f}\nMassimo: {massimo:.2f}',
                        xy=(1.05, 0.5), xycoords='axes fraction',
                        fontsize=12, ha='left', va='center')

30         # Nascondi l'asse X
           plt.gca().xaxis.set_visible(False)

           plt.tight_layout()
           plt.savefig(nome_file)

35         plt.close()

           # Creare e salvare i grafici
           grafici = [
               (timestamps, temperatura, 'Temperatura [ C ]',
                temperatura_media, temperatura_minimo, temperatura_massimo, '
40             grafico_temperatura.png'),
               (timestamps, pressione, 'Pressione [Pa]', pressione_media,
                pressione_minimo, pressione_massimo, 'grafico_pressione.png'),
               (timestamps, luminosita, 'Luminosit [Lumen]',
                luminosita_media, luminosita_minimo, luminosita_massimo, '
45             grafico_luminosita.png')
           ]

           for timestamps, valori, parametro, media, minimo, massimo,
               nome_file in grafici:
               crea_grafico(timestamps, valori, parametro, media, minimo,
50                 massimo, nome_file)

           # Invia i grafici al bot Telegram
           for _, _, _, _, _, _, nome_file in grafici:
               await context.bot.send_photo(chat_id=user.id, photo=open(
                   nome_file, 'rb'))
               os.remove(nome_file)
```

Listing 27: Calcolo statistiche

Di seguito si possono vedere degli esempi dei grafici ottenuti:

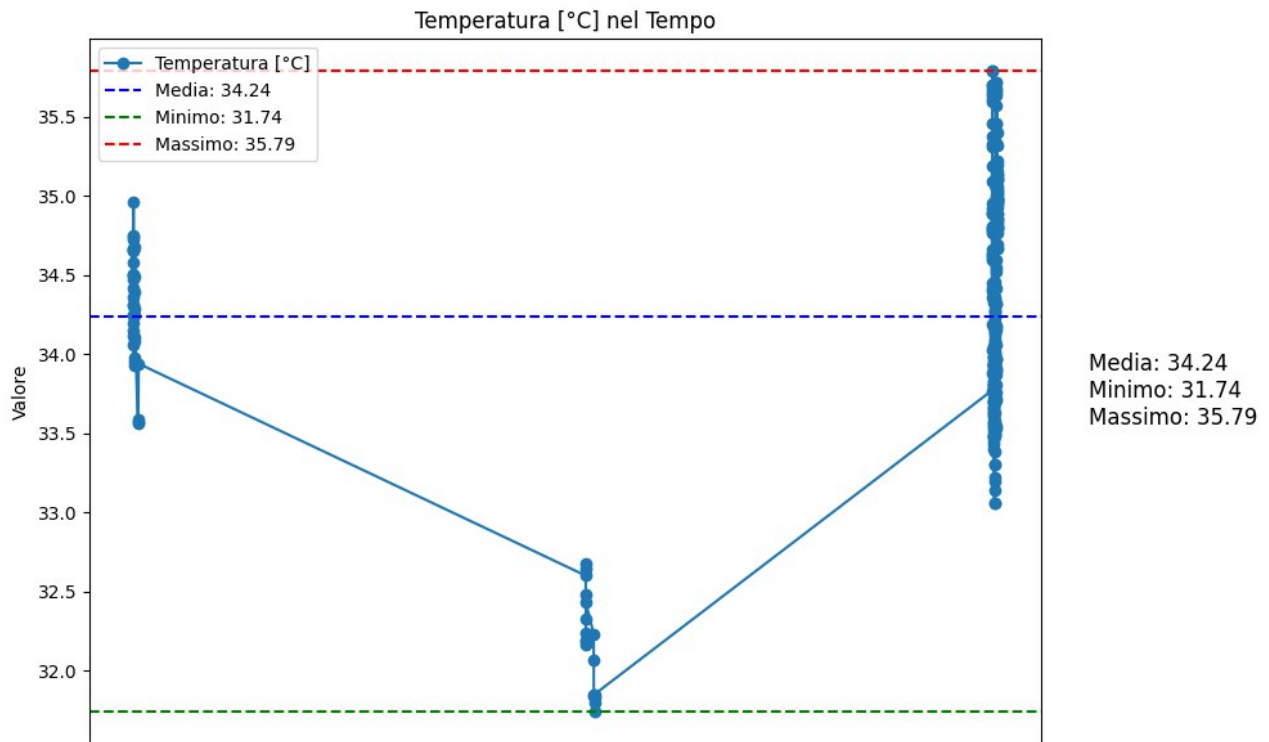


Figura 19: Grafico andamento temperatura

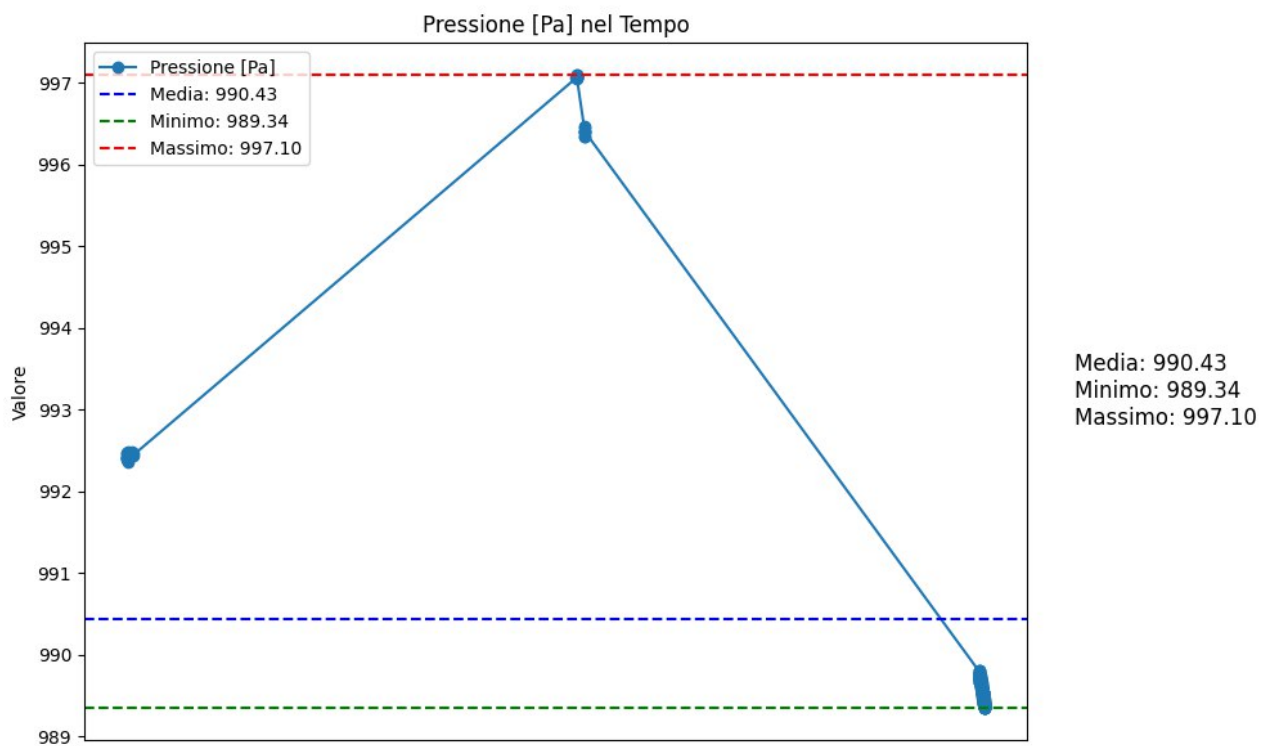


Figura 20: Grafico andamento pressione

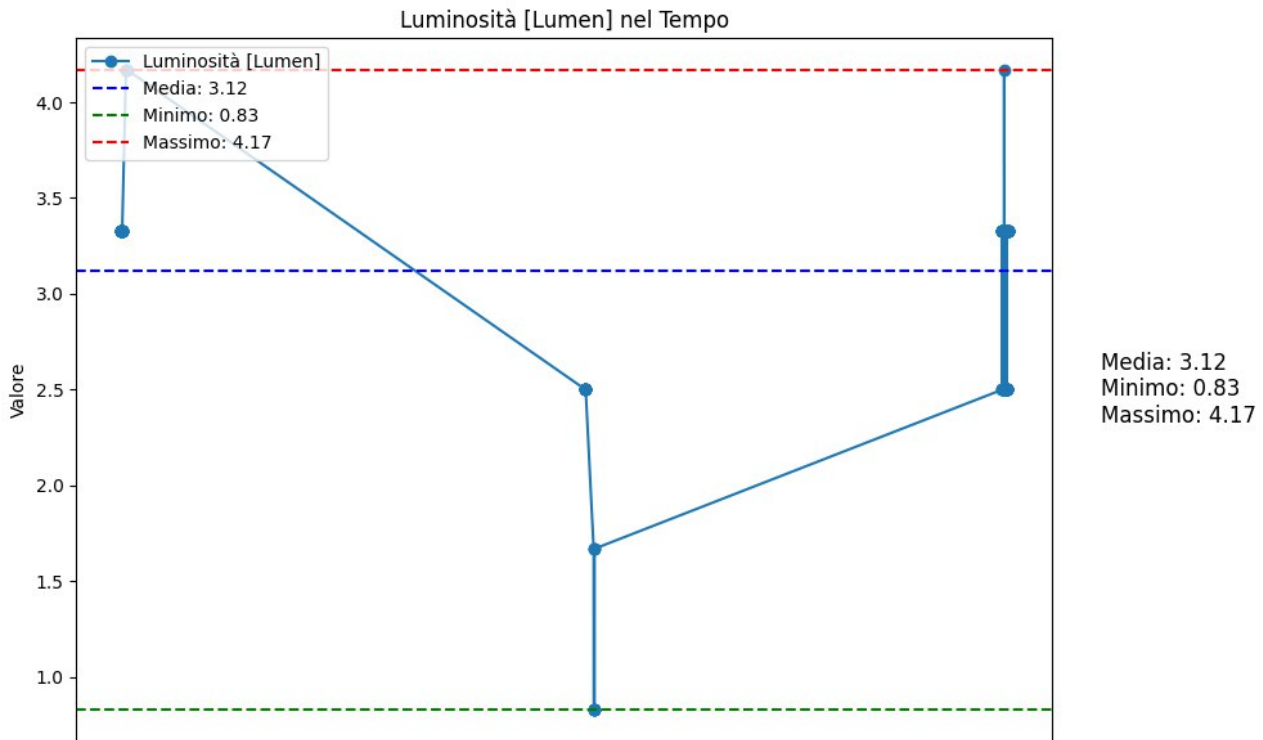


Figura 21: Grafico andamento luminosità

Per poter gestire correttamente tutte le funzionalità, all'interno del **main** devono essere richiamate le funzioni per la gestione dei comandi, per l'avvio del bot e per l'esecuzione del client MQTT in un thread separato.

Quindi, prima di tutto si crea l'applicazione Telegram tramite la funzione

```
application = Application.builder().token(TOKEN).build()
```

a cui viene passato il token identificativo del bot.

Per consentire la gestione dei comandi `/start` e `/leggi` si inserisce la funzione `add_handler()` che prende come argomenti il comando e la funzione di callback.

```
application.add_handler(CommandHandler("start", start))
application.add_handler(CommandHandler("leggi", leggi))
```

Dopodiché si inizializza e si avvia il bot Telegram tramite la funzione `run_polling()`:

```
application.run_polling(allowed_updates=Update.ALL_TYPES)
```

Il client MQTT viene gestito in un thread separato, così da consentire contemporaneamente sia l'ascolto dei messaggi MQTT che la gestione dei comandi provenienti dal bot Telegram.

Viene quindi creato un nuovo thread per eseguire `run_mqtt_client()` il quale richiama

```
sniff_mqtt_messages(broker_address, topics)
```

per gestire il client MQTT indipendentemente dal bot.

Tramite `mqtt_thread.start()` si avvia l'esecuzione del thread, che verrà eseguito in parallelo al thread principale.

3 Sviluppo Progetto

Richiamando la funzione `main()` viene avviata l'applicazione del bot Telegram.

```
def main():
    tracemalloc.start()

    application = Application.builder().token(TOKEN).build()

5    # Gestione del comando /start
    application.add_handler(CommandHandler("start", start))

    # Gestione del comando read
10    application.add_handler(CommandHandler("leggi", leggi))

    # Avvia il bot
    application.run_polling(allowed_updates=Update.ALL_TYPES)

15 # Esegui il client MQTT in un thread separato
def run_mqtt_client():
    sniff_mqtt_messages(broker_address, topics)

if __name__ == '__main__':
20    mqtt_thread = threading.Thread(target=run_mqtt_client)
    mqtt_thread.start()
    main()
```

Listing 28: Definizione istruzioni main

4 Considerazioni finali

Durante lo sviluppo del progetto non abbiamo riscontrato difficoltà rilevanti.

In particolare, studiando i vari sensori, abbiamo notato che il sensore BMP280 non rileva l'umidità, ma solo temperatura e pressione.

Per permettere all'utente di inviare richieste dal bot Telegram, avevamo previsto l'utilizzo della libreria UniversalBotTelegram, ma in fase di test, nonostante la richiesta venisse inviata correttamente alle API di Telegram, non riuscivamo ad ottenere una risposta.

Per questo motivo, è stata utilizzata la libreria HTTPClient.

Si è scelto di implementare il database su una macchina Windows, in quanto sul Raspberry abbiamo riscontrato problemi con i permessi Root.

Infine, avendo posizionato i sensori all'interno di una stanza, non siamo riusciti ad ottenere valori che si discostano molto tra loro.

Con questo progetto è stato realizzato un sistema che permette all'utente di monitorare l'ambiente circostante, controllando i dati rilevati dai vari sensori. L'utilità e le funzionalità fornite, integrate con altri dispositivi, possono essere impiegate quotidianamente sia a casa che negli ambienti lavorativi.